Deep Links

Michał Dubrowski, Piotr Rajski, Mateusz Zawiślak

Czym są deep linki?

Bezpośrednia nawigacja

Przenoszą użytkownika do konkretnego miejsca wewnątrz aplikacji, omijając ekran główny.



To adres wskazujący konkretny pokój w budynku, a nie tylko jego drzwi wejściowe.

Przykłady zastosowań



Zakupy

Klikasz SMS i otwierasz aplikację sklepu dokładnie na karcie promowanego produktu.



Social Media

Kliknięcie linku uruchamia Instagram i natychmiast odtwarza przesłaną rolkę.



Muzyka

System przełącza do odtwarzacza dokładnie w miejscu wybranego podcastu.

Pobierz aplikację testową

Przygotowaliśmy podatną aplikację do pokazania ataków w praktyce.



Po co to?

Dzięki temu rozwiązaniu nie musisz samodzielnie przeklikiwać się przez menu i wyszukiwarkę.

SKANUJ QR

QR do pobrania aplikacji

Do czego się je wykorzystuje?

Marketing
Przenoszenie z reklamy prosto na produkt.

Powiadomienia Push
Otwieranie konkretnej wiadomości.

Udostępnianie treści
Szybkie dzielenie się profilami i rolkami.

Onboarding
Kierowanie z poleceń prosto do sklepu po apkę.

SKANUJ QR

QR do pobrania aplikacji

Jak działają deep linki

1 Praca programisty

Wpisanie przedrostka do pliku konfiguracyjnego.

2 Instalacja

System skanuje plik i dodaje instrukcję do swojego rejestru.

3 Kliknięcie

Przeglądarka wysyła sygnał, a system wybudza odpowiednią aplikację.

SKANUJ QR

QR do pobrania aplikacji

Przykład działania



Przykładowe działanie: victimapp://home

SKANUJ QR



QR do pobrania aplikacji

Przykład implementacji

AndroidManifest.xml

```
<activity
    android:name=".MainActivity"
    android:exported="true"
    android:windowSoftInputMode="adjustResize">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>

    <!-- Plain, legitimate deep link to the home screen: victimapp://home
         (shown as the harmless baseline next to the vulnerable deep links). -->
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />

        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />

        <data
            android:scheme="victimapp"
            android:host="home" />
    </intent-filter>
</activity>
```



Wektory ataków na deep linki

Linki Awaryjne

Poniższy kod QR prowadzi do strony z linkami do “ataków”. Jeżeli kody QR na slajdach nie będą działać skorzystać ze storny w tym linku, kod QR do strony będzie znajdował się cały czas w sekcji bocznej



Auth Bypass (Ominięcie autoryzacji)

Na czym polega

Wykorzystanie deep linku jako tylnego wejścia do chronionego obszaru.

Wektor ataku

Aplikacja posiada ukryte ścieżki do panelu admina bez weryfikacji.

Skutek

Natychmiastowy dostęp do wrażliwych funkcji bez podawania haseł.

SKANUJ QR

QR do strony z linkami do ataków

QR do panelu po zalogowaniu



Deep link do aplikacji

victimapp://home

Domyślny punkt wejścia dla zalogowanego użytkownika



Podatność Auth-Bypass

victimapp://account

Ominięcie weryfikacji sesji pozwalające na bezpośredni dostęp

SKANUJ QR



QR do strony z linkami do ataków

Analiza ataku Auth Bypass

```xml
<!-- Auth Bypass: reachable via victimapp://account with no session check. -->
<activity
    android:name=".AccountActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />

        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />

        <data
            android:scheme="victimapp"
            android:host="account" />
    </intent-filter>
</activity>
```

AndroidManifest.xml

AccountActivity.java

```java
public class AccountActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_account);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v,  WindowInsetsCompat insets) -> {...});

        // VULNERABILITY (Auth Bypass): the protected data is shown with no session check,
        // so a deep link (victimapp://account) reaches it even when the user is logged out.
        // The banner below only visualizes the session state; it does NOT gate access.
        showSessionBanner();
    }
```

Analiza ochrony przed atakiem Auth Bypass

AccountActivity.java

```java
public class AccountActivity extends AppCompatActivity {

    //...

    //...
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_account);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v, WindowInsetsCompat insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        // DEFENSE (Auth Bypass fixed): the protected screen checks the session itself,
        // so the deep link cannot reach the data. No session -> go to login and finish.
        SharedPreferences prefs = getSharedPreferences(MainActivity.PREFS, MODE_PRIVATE);
        if (!prefs.getBoolean(MainActivity.KEY_LOGGED_IN, defValue: false)) {
            startActivity(new Intent( packageContext: this, LoginActivity.class));
            finish();
            return;
        }
        showSessionBanner();
    }
```

Open Redirect i Webview XSS

Na czym polega

Użycie bezpiecznej, zaufanej aplikacji do załadowania złośliwej strony.

Wektor ataku

Aplikacja przyjmuje z linku parametr URL i ładuje go w wewnętrznym oknie bądź przeglądarce zewnętrznej.

Skutek

Ofiara ufa prawdziwej aplikacji, a w wewnętrznej lub zewnętrznej przeglądarce ładuje się phishing lub wykrada dane.

SKANUJ QR

QR do strony z linkami do ataków

Open Redirect (Phishing)

Przekierowanie na naszą stronę

Wzorzec URL:

victimapp://redirect?
url=https://deeplink-security-demo.vercel.app/

Przekierowanie na stronę z phishingiem

Złośliwy URL (Phishing):

victimapp://redirect?
url=https://deeplink-security-demo.vercel.app/phishing-demo

SKANUJ QR

QR do strony z linkami do ataków

Analiza ataku Open Redirect

RedirectActivity.java

```java
public class RedirectActivity extends AppCompatActivity {

    5 usages
    private String targetUrl;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_redirect);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v, WindowInsetsCompat insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        TextView urlView = findViewById(R.id.textUrl);
        MaterialButton openButton = findViewById(R.id.buttonOpen);

        Uri data = getIntent().getData();
        targetUrl = data != null ? data.getQueryParameter( key: "url") : null;

        if (targetUrl == null || targetUrl.isEmpty()) {
            urlView.setText("Brak parametru „url w linku.");
            openButton.setEnabled(false);
            return;
        }

        urlView.setText(targetUrl);
        // The button lets the presenter re-trigger the redirect without re-scanning.
        openButton.setOnClickListener( View v -> openInBrowser());

        // VULNERABILITY (Open Redirect): the url parameter is forwarded to an external
        // browser with no validation of scheme, host, port or anything else.
        openInBrowser();
    }
```

AndroidManifest.xml

```xml
<!-- Open Redirect: forwards victimapp://redirect?url=... to an external browser. -->
<activity
    android:name=".RedirectActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />

        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />

        <data
            android:scheme="victimapp"
            android:host="redirect" />
    </intent-filter>
</activity>
```

Analiza ochrony przed atakiem Open Redirect

RedirectActivity.java

```java
public class RedirectActivity extends AppCompatActivity {
    //...
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_redirect);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v, WindowInsetsCompat insets) -> {...});

        TextView urlView = findViewById(R.id.textUrl);
        MaterialButton openButton = findViewById(R.id.buttonOpen);

        Uri data = getIntent().getData();
        targetUrl = data != null ? data.getQueryParameter( key: "url") : null;

        if (targetUrl == null || targetUrl.isEmpty()) {
            urlView.setText(R.string.redirect_no_url);
            openButton.setEnabled(false);
            return;
        }

        urlView.setText(targetUrl);

        // DEFENSE (Open Redirect fixed): open only URLs that pass the allowlist.
        if (isAllowedRedirect(targetUrl)) {
            openButton.setOnClickListener( View v -> openInBrowser());
            openInBrowser();
        } else {
            openButton.setEnabled(false);
            Toast.makeText( context: this, R.string.redirect_blocked, Toast.LENGTH_LONG).show();
        }
    }

    // DEFENSE: strict allowlist — HTTPS only, exact host, no userinfo, default/443 port.
    // No contains()/endsWith(): those would accept subdomain and suffix traps.
    1 usage
    private boolean isAllowedRedirect(String rawUrl) {
        Uri uri = Uri.parse(rawUrl);
        if (!"https".equals(uri.getScheme())) return false;
        if (uri.getUserInfo() != null) return false;
        if (!"example.com".equals(uri.getHost())) return false;
        int port = uri.getPort();
        return port == -1 || port == 443;
    }
```

WebView XSS

Deep link do aplikacji

Przykład klasycznego użycia parametru

victimapp://xss?name=Michał Kowalski

Podatność WebView XSS

Przykład wykorzystania parametru do ataku Webview XSS:

victimapp://xss?name=<img src=x onerror="alert(document.getElementById('secret').textContent)">

SKANUJ QR

QR do strony z linkami do ataków

Analiza ataku XSS

XssActivity.java

```java
public class XssActivity extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {

        String name = null;
        if (getIntent().getData() != null) {
            name = getIntent().getData().getQueryParameter( key: "name");
        }
        if (name == null || name.isEmpty()) {
            name = "Gość";
        }

        String html;
        try {
            // VULNERABILITY: the untrusted "name" is injected into the HTML template
            // without any encoding, so markup inside it is parsed and executed.
            html = readAsset( fileName: "profile.html").replace( target: "{{NAME}}", name);
        } catch (IOException e) {
            Toast.makeText( context: this, "Nie udało się wczytać szablonu profilu.", Toast.LENGTH_SHORT).show();
            return;
        }

        // Local render only; the base URL just gives the page an https-like origin.
        // Uses a reserved .example host so the injected <img src=x> fails DNS fast
        // (avoids the slow mDNS lookup of a .local host before onerror fires).
        webView.loadDataWithBaseURL( baseUrl: "https://victim.example/", html,  mimeType: "text/html",  encoding: "UTF-8",  historyUrl: null);
    }
```

AndroidManifest.xml

```xml
<!-- WebView XSS: injects victimapp://xss?name=... into a local HTML template. -->
<activity
    android:name=".XssActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />

        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />

        <data
            android:scheme="victimapp"
            android:host="xss" />
    </intent-filter>
</activity>
```

Omówienie ochrony przed atakiem XSS

XssActivity.java

```java
public class XssActivity extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {

        String name = null;
        if (getIntent().getData() != null) {
            name = getIntent().getData().getQueryParameter( key: "name");
        }
        if (name == null || name.isEmpty()) {
            name = getString(R.string.xss_default_name);
        }

        String html;
        try {
            // DEFENSE (WebView XSS fixed): HTML-encode the untrusted value before
            // injecting it, so any markup becomes inert text instead of executing.
            html = readAsset( fileName: "profile.html").replace( target: "{{NAME}}", escapeHtml(name));
        } catch (IOException e) {
            Toast.makeText( context: this, R.string.xss_template_error, Toast.LENGTH_SHORT).show();
            return;
        }
        webView.loadDataWithBaseURL( baseUrl: "https://victim.example/", html, mimeType: "text/html", encoding: "UTF-8", historyUrl: null);
    }

    // DEFENSE: minimal HTML entity encoding for untrusted text.
    1 usage
    private String escapeHtml(String s) {
        return s.replace( target: "&", replacement: "&amp;")
                .replace( target: "<", replacement: "&lt;")
                .replace( target: ">", replacement: "&gt;")
                .replace( target: "\"", replacement: "&quot;")
                .replace( target: "'", replacement: "&#39;");
    }
```



Rodzaje deep linków

Standardowe Deep Linki

Unikalny przedrostek

Używają własnego schematu (np.).`victimapp://`

Działanie lokalne

System po prostu szuka zadeklarowanej aplikacji w rejestrze.

Główna wada: Brak weryfikacji

Nikt nie sprawdza, czy dany program ma "prawo własności" do tego adresu.

SKANUJ QR

QR do strony z linkami do ataków

Deferred Deep Links

Brak aplikacji

Przekierowanie niezainstalowanych użytkowników do sklepu (App Store / Google Play).

Pamięć kontekstu

Automatyczne otwarcie docelowego ekranu po instalacji i pierwszym uruchomieniu.

Zastosowanie

Płynny onboarding z kampanii reklamowych oraz programów poleceń.

SKANUJ QR

QR do strony z linkami do ataków

Deep Link Hijacking

Na czym polega

Złośliwa aplikacja "porywa" link przeznaczony dla bezpiecznej aplikacji.

Wektor ataku

Haker tworzy fałszywą aplikację rejestrującą ten sam schemat co apka docelowa.

Skutek (Phishing)

Złośliwa apka przechwytuje link, wyświetla fałszywy panel i kradnie dane logowania.

SKANUJ QR

QR do strony z linkami do ataków

Pobierz złośliwą aplikację testową

Przygotowaliśmy podatną aplikację do pokazania ataków w praktyce.



Link do Ataku

SKANUJ QR

QR do strony z linkami do ataków

⚠WAŻNA UWAGA!

Kiedy system spyta Cię o wybór aplikacji

WYBIERZ AttackerAPP

Co można z tym zrobić?

Ulepszone deep linki - App Links i Universal Links

Moment instalacji

System w tle wysyła zapytanie na serwer domeny w poszukiwaniu pliku `.assetlinks.json`

Kryptograficzny dowód

Zgodność certyfikatów autoryzuje własność domeny. Eliminuje to Hijacking.

SKANUJ QR

QR do strony z linkami do ataków

Prezentacja App Links

```
<!-- Good practice (Android App Links): verified HTTPS link cannot be hijacked.
     Host must match the deployed demo-site domain and its assetlinks.json. -->
<activity
    android:name=".SecureActivity"
    android:exported="true">
    <intent-filter android:autoVerify="true">
        <action android:name="android.intent.action.VIEW" />

        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />

        <data
            android:scheme="https"
            android:host="deeplink-security-demo.vercel.app"
            android:pathPrefix="/secure" />
    </intent-filter>
</activity>
```

AccountActivity.java

deeplink-security-demo.vercel.app/.well-known/assetlinks.json


```
[
  {
    "relation": [
      "delegate_permission/common.handle_all_urls"
    ],
    "target": {
      "namespace": "android_app",
      "package_name": "com.example.victimapp",
      "sha256_cert_fingerprints": [
        "B8:69:D2:F4:B9:D3:91:84:40:54:6D:C9:49:1F:61:01:6B:47:39:1F:36:BD:78:F6:68:2A:DC:6F:25:22:C4:70"
      ]
    }
  }
]
```



Cyberbezpieczeństwo a deep links

Testowanie deep linków

SAST (Analiza statyczna)

- Audyt pliku Manifestu
- Weryfikacja flagi `android:exported`
- Statyczna analiza przepływu danych

DAST (Analiza dynamiczna)

- Testy w środowisku wykonawczym
- Instrumentacja aplikacji
- Weryfikacja bezpiecznego ładowania domen WebView

OWASP i MASTG

Open Worldwide Application Security Project & Mobile Application Security Testing Guide

IPC Input Validation

Weryfikacja i bezpieczna obsługa danych wejściowych przekazywanych pomiędzy procesami aplikacji.

Open URL Redirection

Ochrona przed nieautoryzowanym i niebezpiecznym przekierowywaniem użytkowników na zewnętrzne adresy URL.

Custom URL Scheme Hijacking

Zabezpieczenie niestandardowych schematów URL przed ich przejęciem przez złośliwe aplikacje.

Gdzie szukać informacji?

Kluczowe rejestry i bazy danych dotyczące podatności bezpieczeństwa

Rejestr CVE

Common Vulnerabilities and Exposures (CVE-[ROK]-[NUMER]).

Standardowy Format Identyfikatora

`CVE-[ROK]-[NUMER]`

Baza NVD

National Vulnerability Database oceniająca skalę krytyczności CVSS (0.0 - 10.0).

Skala Krytyczności CVSS

`0.0 – 10.0`

Najbardziej znane ataki

Przykłady krytycznych podatności bezpieczeństwa w popularnych aplikacjach

TikTok

CVE-2022-28799

1-Click Account Takeover.

Zoom

CVE-2026-53407

Custom URL Handler Breach.

KakaoTalk

CVE-2023-51219

WebView JavaScript Execution.

Dobre praktyki

Kluczowe zasady bezpieczeństwa przy wdrażaniu rozwiązań

Universal Links

Weryfikacja domen

Wdrażaj tylko weryfikowalne App/Universal Links.

Allowlisty WebView

Kontrola dostępu

Stosuj sztywne allowlisty dla WebView.

Tokeny Sesji

Zero Trust

Nigdy nie ufaj wejściom bez weryfikacji tokenów sesji.



Jak nie dać się złapać?

Kluczowe zasady bezpieczeństwa podczas korzystania z linków i aplikacji

Zasada ograniczonego zaufania

Ekrany logowania

Jeśli otworzona linkiem aplikacja prosi o zalogowanie lub podanie danych – zignoruj to, zamknij ją i otwórz ręcznie z ekranu głównego.

Weryfikacja poza linkiem

Komunikaty SMS i wiadomości

Jeśli SMS z linkiem straszy niedopłatą do paczki lub blokadą konta – zignoruj link i sprawdź status sprawy ręcznie w oficjalnej aplikacji.

Kontrola wyboru aplikacji

Okno „Otwórz za pomocą...”

Jeśli telefon wyświetla zapytanie o wybór aplikacji – nie klikaj odruchowo „Zawsze”, dopóki nie upewnisz się, że wybierasz sprawdzony program.

Bieżące aktualizacje oprogramowania

Ochrona przed atakami 1-Click

Jeśli system lub sklep z aplikacjami powiadamia o aktualizacji – zainstaluj ją niezwłocznie. Nowe łatki blokują luki logiczne wykorzystywane do przejmowania kont i wzmacniają weryfikację bezpiecznych linków.

Pytania

Dziękujemy za uwagę!